

Plan de Pruebas de Software

codebymike.tech

Fecha: 26-08-2026

Tabla de contenido

Historial de Versiones	4
Información del Proyecto	4
Aprobaciones	4
Resumen Ejecutivo	5
Alcance de las Pruebas	5
Elementos de Pruebas	5
Nuevas Funcionalidades a Probar	6
Pruebas de Regresión	6
Funcionalidades a No Probar	7
Enfoque de Pruebas (Estrategia)	7
Criterios de Aceptación o Rechazo	8
Criterios de Aceptación o Rechazo	8
Criterios de Suspensión	8
Criterios de Reanudación	9
Entregables	9
Recursos	10
Requerimientos de Entornos – Hardware	10
Requerimientos de Entornos – Software	10
Herramientas de Pruebas Requeridas	11
Personal	11
Entrenamiento	12
Planificación y Organización	12
Procedimientos para las Pruebas	12

Matriz de Responsabilidades	13
Cronograma	13
Premisas	14
Dependencias y Riesgos	14
Referencias	15
Glosario	15

Historial de Versiones

Fecha	Versión	Autor	Organización	Descripción
26-08-2026	1.0	Mike	codebymike.tech	Versión inicial del plan de pruebas, adaptado de la plantilla SENA a la estrategia de testing ya implementada en el repositorio.

Información del Proyecto

Empresa / Organización	codebymike.tech
Proyecto	Portafolio + panel de control + portal de clientes + laboratorio de ingeniería
Fecha de preparación	26-08-2026
Cliente	SENA (sustentación) / proyecto propio
Patrocinador principal	Mike
Gerente / Líder de Proyecto	Mike
Gerente / Líder de Pruebas de Software	Mike

Aprobaciones

Nombre y Apellido	Cargo	Departamento u Organización	Fecha	Firma
Mike	Desarrollador / Líder de Pruebas	codebymike.tech	26-08-2026	Mike

Resumen Ejecutivo

El plan de pruebas de codebymike.tech (portafolio, panel de administración, portal de clientes y laboratorio de ingeniería) cubre la estrategia de calidad ya implementada y operando en producción: pruebas unitarias y de integración con Vitest, pruebas end-to-end con Playwright, mutation testing con Stryker, análisis estático y dinámico de seguridad (SAST/DAST) y accesibilidad automatizada, todo integrado al pipeline de CI/CD de GitHub Actions y reportado en el propio panel del laboratorio (/admin/lab).

Alcance de las Pruebas

Cobertura de lógica de negocio pura (src/lib/: pagos, portal de clientes, seguridad, i18n), APIs (src/pages/api/), y flujos end-to-end críticos del sitio público, el portal y el panel de administración.

Módulos, Componentes o Elementos a Probar:

Lógica pura (pagos, portal, micro-SIEM, i18n), APIs de admin/portal/cron/pagos, flujos e2e (contacto, portal, público, demo, autenticación, pagos), seguridad de dependencias y código, seguridad dinámica de la app desplegada, y accesibilidad de páginas públicas.

Nuevas Funcionalidades a Probar

Cada feature nueva incorpora sus propios tests unitarios (y e2e cuando aplica el flujo) antes de mergeare a main: ningún cambio en src/lib/ se considera terminado sin su test en Vitest.

Pruebas de Regresión

La suite completa de Vitest y Playwright corre en cada push y cada pull request contra main (ci.yml), evitando que un cambio nuevo rompa funcionalidad ya validada.

Funcionalidades a No Probar

Nunca se ejecuta carga ni DAST contra codebymike.tech (producción); las integraciones opcionales (ntfy, Resend) no se prueban contra el proveedor real cuando falta su credencial, por diseño (no-op silencioso).

Enfoque de Pruebas (Estrategia)

Pirámide de pruebas: unitarias (Vitest) como base, e2e (Playwright) para flujos críticos, y una capa transversal de mutación, SAST, DAST y accesibilidad que corre en el pipeline de CI/CD.

Criterios de Aceptación o Rechazo

La suite de Vitest y Playwright debe estar 100% en verde antes de mergear a main; el deploy a producción solo se acepta si /api/health responde sano tras el despliegue.

Criterios de Suspensión

Un hallazgo crítico de seguridad (SAST/DAST) o un healthcheck fallido tras el deploy suspenden la promoción de esa versión.

Criterios de Reanudación

Se retoma el pipeline en el siguiente push a main una vez corregido el hallazgo o la regresión.

Entregables

Suite de Vitest (tests/) y Playwright (e2e/), reportes de cobertura, hallazgos de seguridad/accesibilidad en /admin/lab/security y /lab, score de mutación cuando se ejecuta, y logs de cada workflow en GitHub Actions.

Recursos

Infraestructura serverless (sin hardware físico dedicado) y herramientas de código abierto integradas al pipeline.

Requerimientos de Entornos – Hardware

No aplica hardware físico dedicado: runners de GitHub Actions (ubuntu-latest) para CI, Vercel Functions (Fluid Compute) para producción, y Turso/libSQL como base de datos.

Requerimientos de Entornos – Software

Node.js >=22.12, Vitest, Playwright, Stryker, npm audit, CodeQL, OWASP ZAP, axe-core, Drizzle ORM.

Herramientas de Pruebas Requeridas

Vitest y Playwright para automatización; el propio panel /admin/lab/* como software de gestión de resultados de pruebas.

Personal

Mike: líder de pruebas, analista y especialista en automatización (proyecto de una sola persona, varios roles).

Entrenamiento

No aplica entrenamiento externo: la infraestructura de pruebas fue diseñada e implementada por el mismo desarrollador que la opera.

Planificación y Organización

Las pruebas no siguen un cronograma de proyecto único: corren de forma continua, disparadas por eventos del pipeline (push, PR, cron semanal, o manualmente).

Procedimientos para las Pruebas

TDD parcial en lógica pura de src/lib/; BD real en archivo temporal de libSQL cuando hay concurrencia o restricciones UNIQUE de por medio.

Matriz de Responsabilidades

Una sola persona ejerce todos los roles de QA; ver el RACI detallado en la Matriz de Responsabilidades del capítulo siguiente.

Cronograma

Cadencia continua ligada a eventos de CI/CD, no a fechas fijas de un proyecto con fin determinado; ver el detalle en el capítulo de Cronograma.

Premisas

El pipeline de GitHub Actions y el auto-deploy de Vercel son el ambiente de referencia; los secrets opcionales degradan con gracia si faltan.

Dependencias y Riesgos

Principal riesgo: dependencia de VERCEL_TOKEN para el rollback automático, y cuota de lecturas de Turso si una prueba apunta por error a la base de datos real.

Referencias

docs/plan-lab.md, docs/plan-lab-fases-pendientes.md, CLAUDE.md.

Alcance de las Pruebas

El plan cubre la estrategia de pruebas automatizadas del proyecto codebymike.tech: portafolio público, panel de administración privado, portal de clientes y laboratorio de ingeniería. Se prueban tanto los componentes de software (lógica pura, APIs, páginas) como los procesos de calidad que rodean cada despliegue (CI/CD, seguridad, accesibilidad).

Módulos, Componentes o Elementos a Probar:

Lógica pura (src/lib/): pagos (máquina de estados en payments-state.ts, idempotencia en payments.ts), portal de clientes (sesiones, aislamiento por clientId), seguridad (micro-SIEM: clasificador de amenazas, rate limiting durable, blocklist), i18n (diccionarios y enrutamiento).

APIs (src/pages/api/): admin (bajo sesión admin), portal (bajo sesión de cliente), cron (bearer CRON_SECRET), pagos.

Flujos end-to-end (Playwright, e2e/): contact.spec.ts, portal.spec.ts, public.spec.ts, demo.spec.ts, auth.spec.ts, payments.spec.ts.

Seguridad estática (SAST): dependencias (npm audit) y código (CodeQL).

Seguridad dinámica (DAST): OWASP ZAP baseline contra el preview deployment de cada PR.

Accesibilidad (a11y): axe-core sobre las páginas públicas.

Nuevas Funcionalidades a Probar

Regla del repo (CLAUDE.md, «Al añadir una feature grande»): si toca dinero o datos personales se revisa primero si payments.ts, lib/security/* o lib/notify.ts ya resuelven parte del problema; toda migración de schema es aditiva; se agregan tests de lógica pura primero y de integración con libSQL temporal si hay concurrencia o UNIQUE de por medio; una feature grande termina con su caso de estudio en src/content/notes/.

Pruebas de Regresión

ci.yml corre test (Vitest) y build en cada push y cada pull request; en main, además espera el auto-deploy de Vercel, lo verifica contra /api/health y ejecuta rollback automático si la nueva versión no responde sana. La regresión no se limita al código: también cubre la salud del propio despliegue.

Funcionalidades a No Probar

Carga (k6) y DAST nunca se dirigen a codebymike.tech: el guardarraíl de lab/k6/lib/perfil.js aborta cualquier corrida cuya URL o base de datos apunte a producción, y el workflow de DAST usa siempre la URL del preview deployment de GitHub. Motivo: Vercel factura por invocación y CPU activa, y Turso tiene cuota de filas en el plan gratuito, y un incidente real de agosto de 2026 agotó la cuota de lecturas al correr k6 sin este guardarraíl.

Enfoque de Pruebas (Estrategia)

Tipo de prueba	Herramienta	Disparador	Frecuencia
Unitarias	Vitest	push / pull request	cada cambio (ci.yml)
E2E	Playwright	push / pull request	cada cambio (ci.yml, test:e2e:server)
SAST	npm audit + CodeQL	push / PR + cron	cada cambio y domingos 06:00 UTC (security.yml)
DAST	OWASP ZAP baseline	pull request	cada PR, contra preview deployment (dast.yml)
Accesibilidad	axe-core	push / pull request	cada cambio (a11y.yml)
Mutación	Stryker	workflow_dispatch + cron	domingos 08:00 UTC (mutation.yml)
Verificación de	ci.yml + Vercel	push a main	cada deploy a

deploy + rollback

producción

Tipo de prueba, herramienta, disparador y frecuencia de cada capa de la estrategia:

Criterios de Aceptación o Rechazo

Criterios de Aceptación o Rechazo

100% de los 898 tests de Vitest y 40 tests e2e de Playwright en verde; cobertura medida con `npm run test:coverage` (sin gate estricto de porcentaje, pero se revisa en cada feature grande); `/api/health` debe responder sano tras cada deploy a producción para que `ci.yml` considere el release aceptado.

Criterios de Suspensión

Un hallazgo de severidad crítica en `npm audit/CodeQL` (SAST) o en el baseline de ZAP (DAST) bloquea el merge hasta remediarlo o aceptarlo explícitamente como riesgo conocido en `/admin/lab/security`. Si `/api/health` falla tras un deploy a main, `ci.yml` ejecuta rollback automático (requiere `VERCEL_TOKEN`) y notifica por ntfy si está configurado.

Criterios de Reanudación

Se reanuda con el siguiente push a main una vez corregido el hallazgo o revertido el cambio que causó la regresión; el pipeline completo (test, build, deploy, healthcheck) vuelve a correr desde cero.

Entregables

Suite de Vitest: `tests/*.test.ts` (898 tests).

Suite de Playwright: `e2e/*.spec.ts` (40 tests, 6 specs).

Reporte de cobertura: `npm run test:coverage`.

Hallazgos de seguridad y accesibilidad: `/admin/lab/security` (privado) y `/lab` (agregado público).

Score de mutación: reportado a `/admin/lab` cuando corre `mutation.yml`.

Logs y artefactos de cada ejecución: pestaña Actions de GitHub.

Recursos

Ver detalle en las subsecciones de Hardware, Software y Herramientas.

Requerimientos de Entornos – Hardware

El proyecto no contrata ni administra hardware físico. La ejecución de pruebas usa: runners ubuntu-latest de GitHub Actions (CI, SAST, DAST, a11y, mutación), Vercel Functions con Fluid Compute (producción y previews), y Turso/libSQL (base de datos, con instancia aparte para pruebas de demo). En desarrollo local, compose.yaml levanta dos servidores sqld (Docker) para pruebas contra libSQL real.

Requerimientos de Entornos – Software

Node.js >=22.12 (requerido por package.json engines), Vitest (unitarias), Playwright (e2e), Stryker (mutación), npm audit y CodeQL (SAST), OWASP ZAP baseline (DAST), axe-core (a11y), Drizzle ORM y drizzle-kit (migraciones), sqld/libSQL (Docker, solo desarrollo).

Herramientas de Pruebas Requeridas

Automatización: Vitest + Playwright + Stryker, orquestados por GitHub Actions. Gestión y visualización de resultados: panel propio /admin/lab/* (no se usa una herramienta de gestión de pruebas de terceros), alimentado por POST /api/lab/ingest con autenticación Bearer LAB_INGEST_TOKEN.

Personal

Mike (@mikerb95) ejerce los tres roles del equipo de QA: líder de pruebas (define alcance y prioridades), analista/tester (escribe y ejecuta los casos), y especialista en automatización (mantiene Vitest, Playwright, Stryker y los workflows de CI/CD). No hay separación de personas por rol: es un proyecto de un solo desarrollador.

Entrenamiento

No se requiere entrenamiento externo: la infraestructura de pruebas (Vitest, Playwright, Stryker, SAST/DAST, a11y) fue diseñada, implementada y documentada por el mismo desarrollador que la opera, como parte del roadmap de ingeniería documentado en docs/plan-lab.md.

Planificación y Organización

A diferencia de un proyecto con fecha de cierre, las pruebas de codebymike.tech corren de forma continua: cada evento del pipeline (push, pull request, cron semanal o disparo manual) activa el subconjunto de pruebas correspondiente.

Procedimientos para las Pruebas

Para lógica pura en src/lib/ se prioriza escribir o actualizar el test de Vitest junto con el cambio, usando BD real en archivo temporal (tmpdir()), nunca :memory:) cuando hay concurrencia o restricciones UNIQUE de por medio (ver tests/payments.test.ts como plantilla). Los e2e de Playwright siembran bases libSQL desechables en webServer.command, nunca en globalSetup. Un módulo que se importa desde el navegador no puede depender de node:crypto ni de la capa de BD: la lógica isomorfa se separa en un módulo puro aparte.

Matriz de Responsabilidades

Actividad	R	A	C	I
Diseño de casos de prueba	Mike	Mike	-	-
Ejecución de pruebas unitarias (Vitest)	Mike	Mike	-	-
Ejecución de pruebas e2e (Playwright)	Mike	Mike	-	-
Revisión de hallazgos SAST/DAST	Mike	Mike	-	-
Revisión de accesibilidad (a11y)	Mike	Mike	-	-
Aprobación de merge a main	Mike	Mike	-	-
Rollback ante	Mike	Mike	-	GitHub

fallo de
healthcheck

Actions
(automático)

Al ser un equipo de una sola persona, Mike concentra los roles R y A en toda actividad; se documenta igual para dejar explícita la responsabilidad de cada tarea:

Cronograma

Actividad	Disparador	Frecuencia
Unitarias (Vitest) + build	push / pull request	cada cambio (ci.yml)
E2E (Playwright)	push / pull request	cada cambio (ci.yml, vía test:e2e:server)
SAST (npm audit + CodeQL)	push / pull request + cron	cada cambio y domingos 06:00 UTC (security.yml)
DAST (OWASP ZAP)	pull request	cada PR, contra preview deployment (dast.yml)
Accesibilidad (axe-core)	push / pull request	cada cambio (a11y.yml)
Mutación (Stryker)	manual (workflow_dispatch) + cron	domingos 08:00 UTC (mutation.yml)
Verificación de deploy + rollback	push a main	cada deploy a producción (ci.yml)

Cadencia de cada capa de pruebas, según su disparador en GitHub Actions:

Premisas

Se asume disponibilidad del runner ubuntu-latest de GitHub Actions y del auto-deploy de Vercel como ambiente de referencia para e2e/DAST vía preview deployments. Los secrets opcionales (VERCEL_TOKEN, LAB_INGEST_TOKEN, NTFY_TOPIC) se asumen configurados en producción; su ausencia degrada con gracia (patrón fail-open del repo) pero no bloquea el pipeline. Un solo desarrollador ejecuta todos los roles de QA, sin redundancia de personal.

Dependencias y Riesgos

Riesgos identificados: (1) sin `VERCEL_TOKEN`, un deploy roto no se revierte automáticamente y requiere intervención manual; (2) una prueba de carga o e2e mal dirigida contra la base de datos de producción puede agotar la cuota de filas leídas de Turso en el plan gratuito, mitigado con el guardarraíl de dos mitades de `lab/k6/lib/perfil.js` (verifica URL y que `/api/health` confirme una base local); (3) dependencia de que exista un preview deployment válido para que corran DAST y, en el futuro, load testing.

Referencias

`docs/plan-lab.md` (plan del módulo LAB, fases 0-6), `docs/plan-lab-fases-pendientes.md` (fases 5-7 pendientes), `docs/plan-docker.md` (infraestructura de desarrollo/pruebas), `CLAUDE.md` (convenciones de testing del repo), workflows de GitHub Actions (`ci.yml`, `security.yml`, `dast.yml`, `a11y.yml`, `mutation.yml`).

Glosario

SAST: análisis estático de seguridad (código y dependencias), sin ejecutar la aplicación.

DAST: análisis dinámico de seguridad contra la aplicación desplegada y en ejecución.

E2E (end-to-end): prueba que simula un flujo completo de usuario real sobre la aplicación desplegada.

Mutation testing: técnica que introduce mutaciones deliberadas en el código para medir si la suite de tests las detecta.

CI/CD: integración y despliegue continuos; en este repo, GitHub Actions + auto-deploy de Vercel.

Fail-open: patrón de diseño donde un fallo en un componente de seguridad u observabilidad no bloquea el flujo principal del sistema.

RACI: modelo de asignación de responsabilidades (Responsable, Aprobador, Consultado, Informado).